Open in app ↗

Medium

Access to everything. Now 30% off. Upgrade now

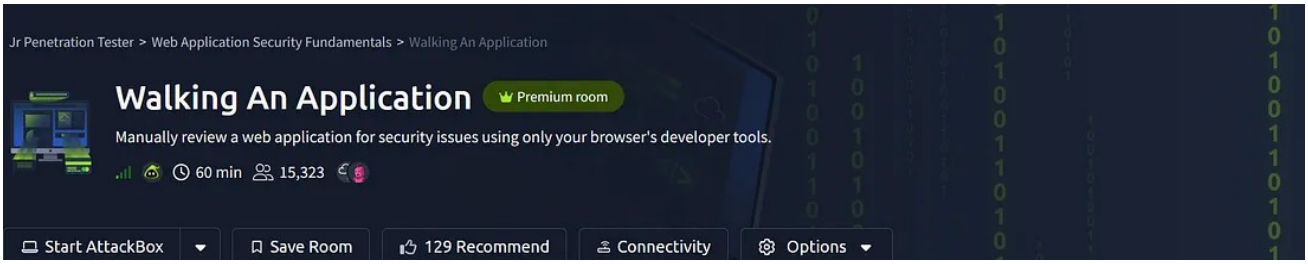
Walking An Application +

13 min read · Just now



Nuh hozar

Listen Share ••• More



jr pen tester Web Application Security Fundamentals

Walking An Application “TRYHACKME WALKTROUGH”

Task 1- Introduction

In this room, you will learn how to manually review a web application for security issues using only the built-in tools in your browser. More often than not, automated security tools and scripts will miss many potential vulnerabilities and useful information.

Learning Objectives

By the end of this room, you will be able to:

- Use your browser to view the human-readable source code of a website
- Inspect page elements and make changes to view the usually blocked content
- Inspect and control the flow of a page’s JavaScript

- Identify the network requests a page makes
- Identify locally stored data by a website

Prerequisites

You should have an understanding of the following rooms before starting:

- Web Application Basics
- HTTP In Detail

Task 2- Exploring the Website

As a penetration tester, your role when reviewing a website or web application is to identify features that could be vulnerable and attempt to exploit them to assess whether they are. These features are usually parts of the website that require user interaction.

Finding interactive portions of the website can range from spotting a login form to manually reviewing the website's JavaScript. An excellent place to start is just with your browser; explore the website, note down the individual pages, areas, and features, and include a summary for each. After visiting the Acme IT Support website, you will see the following page:



An example site review for the Acme IT Support website would look something like this:

Feature	Endpoint	Summary
Home Page	/	This page contains a summary of what Acme IT Support does, along with a company photo of its staff.
Latest News	`/news`	This page contains a list of recently published news articles by the company, and each news article has a link with an ID number, i.e. `/news/article?id=1`
News Article	`/news/article?id=1`	Displays the individual news article. Some articles seem to be blocked and reserved for premium customers only.
Contact Page	`/contact`	This page contains a form for customers to contact the company. It contains name, email, and message input fields, along with a send button.
Customers	`/customers`	This link redirects to `/customers/login`.
Customer Login	`/customers/login`	This page contains a login form with username and password fields.
Customer Signup	`/customers/signup`	This page contains a user signup form with input fields for username, email, password, and password confirmation.
Customer Reset Password	`/customers/reset`	Password reset form with an email address input field.
Customer Dashboard	`/customers`	This page lists the user's submitted tickets to the IT support company and includes a **Create Ticket** button.
Create Ticket	`/customers/ticket/new`	This page contains a form with a textbox for entering the IT issue and a file upload option to create an IT support ticket.
Customer Account	`/customers/account`	This page allows the user to edit their username, email and password.
Customer Logout	`/customers/logout`	This link logs the user out of the customer area.

We will start taking a deeper look into some of the pages we have discovered in the next task.

Q 1-What is the endpoint for creating new tickets?

Answer : /customers/ticket/new

Task 3- **Viewing The Page Source**

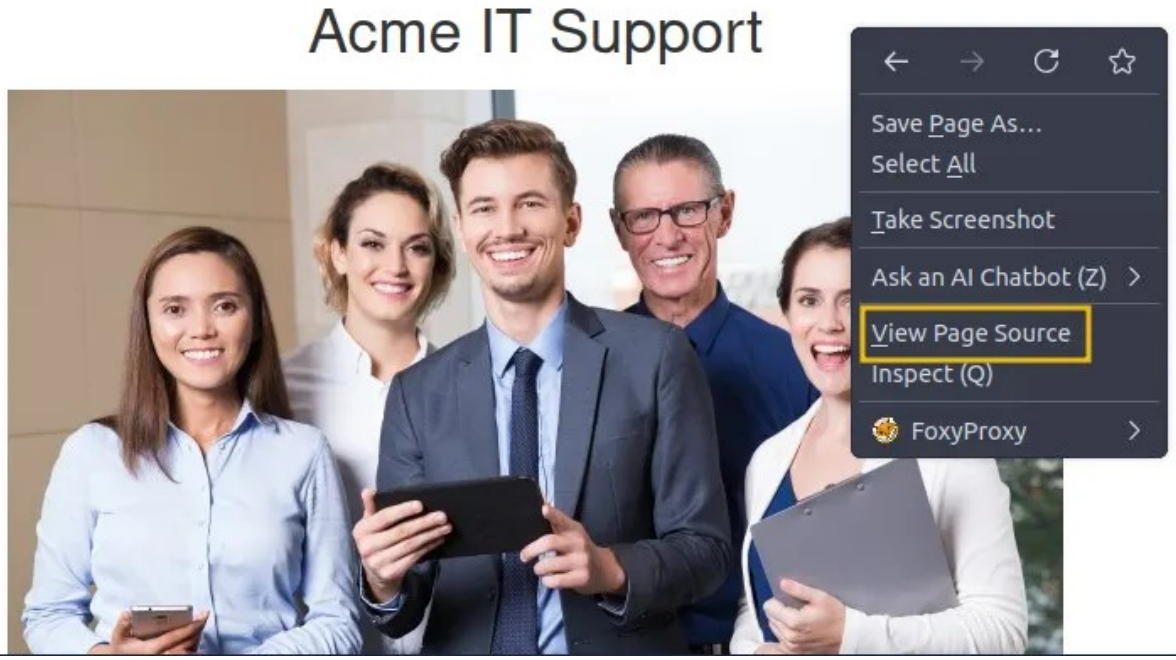
The **page source** is the human-readable code returned to our browser/client from the web server each time we make a request.

The returned code is made up of HTML (HyperText Markup Language), CSS (Cascading Style Sheets), and JavaScript, and it tells our browser what content to display, how to show it, and adds interactivity with JavaScript.

For our purposes, viewing the page source can help us discover more information about the web application.

How Do I View the Page Source?

While viewing a website, you can right-click the page and select **View Page Source** from the menu.



Most browsers also support putting view-source in front of the URL, for example, `view-source:https://www.google.com/` .

Let's View Some Page Source!

Try viewing the source code of the Acme IT Support website's home page. Unfortunately, explaining everything you can see here is well out of the scope of

this room, and you'll need to look into website design/development courses to understand it fully. What we can do is pick out bits of information that are of importance to us.

At the top of the page, you'll notice some code starting with `<!--` and ending with `-->`; these are **comments**. Comments are messages left by the website developer, usually to explain something in the code to other programmers or even notes/reminders for themselves. These comments don't get displayed on the actual webpage. This comment explains that the homepage is temporary while a new one is under development. **View the webpage in the comment to get your first flag.**



Links to different pages in HTML are written in anchor tags (these are HTML elements that start with `<a`), and the link that you'll be directed to is stored in the `href` attribute.

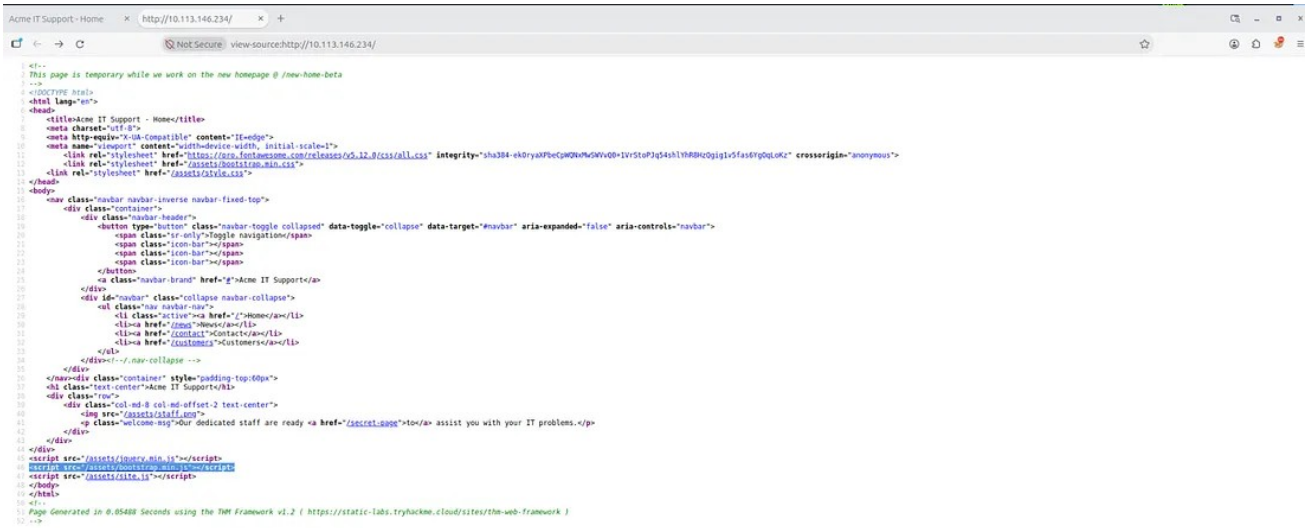
For example, you'll see the contact page link on line `31`:

```
<div id="navbar" class="collapse navbar-collapse">
    <ul class="nav navbar-nav">
        <li class="active"><a href="/">Home</a></li>
        <li><a href="/news">News</a></li>
        <li><a href="/contact">Contact</a></li>
        <li><a href="/customers">Customers</a></li>
    </ul>
</div><!--/.nav-collapse -->
```

If you view the page source further down, there is a hidden link to a page starting with `secr`; **view this link to get another flag.** You obviously wouldn't get a flag in a real-world situation, but you may discover a private area the business uses to store company, staff, or customer information.

External files such as CSS, JavaScript and images can be included using the

HTML code. In this example, you'll notice that these files are all stored in the same directory.



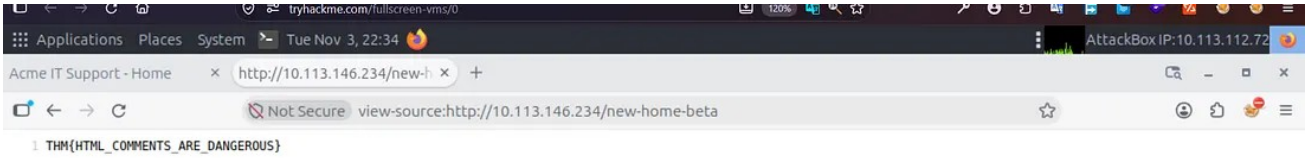
If you view this directory in your web browser, you should see a configuration error: either a blank page or a **403 Forbidden** page with an error stating you don't have access to the directory. Instead, the directory listing feature has been enabled, which actually lists every file in the directory. Sometimes this isn't an issue, and all the files in the directory are safe for public viewing, but in some cases, backup files, source code, or other confidential information could be stored here. **In this instance, there is a flag in the `flag.txt` file.**

Many websites these days aren't built from scratch; they use a **framework.** A framework is a collection of pre-made code that makes it easy for developers to include common features a website would require, such as blogs, user management, form processing, and more, saving developers hours or days of development time.

Viewing the **page source** can often give us clues into whether a framework is in use and, if so, which framework and even what version. Knowing the framework and version can be a powerful find, as there may be public vulnerabilities in the framework, and the website might not be using the most up-to-date version. At the bottom of the page, you'll find a comment about the framework and its version in use, along with a link to the framework's website. Viewing the framework's website, you'll see that our website is, in fact, out of date. **Read the update notice and use the information that you find to discover another flag.**

Q 2- What is the flag from the HTML comment?

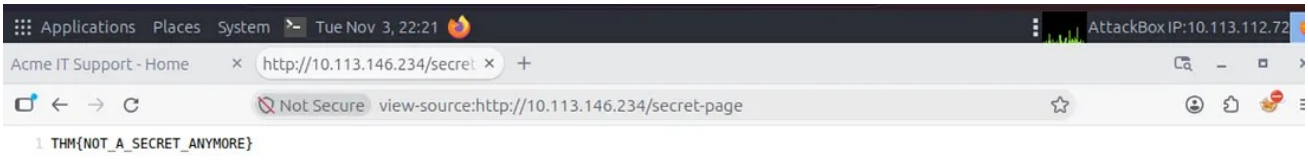
Answer : THM{HTML_COMMENTS_ARE_DANGEROUS}



Q 3- What is the flag from the secret link?

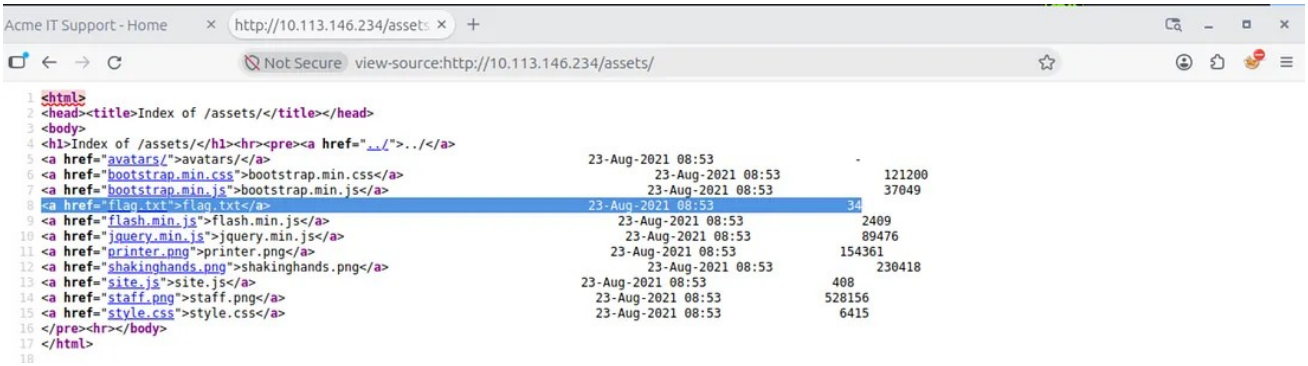
Answer : THM{NOT_A_SECRET_ANYMORE}

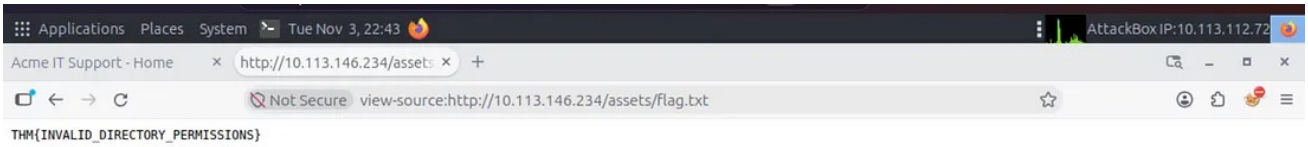




Q 4-What is the directory listing flag?

Answer : THM{INVALID_DIRECTORY_PERMISSIONS}

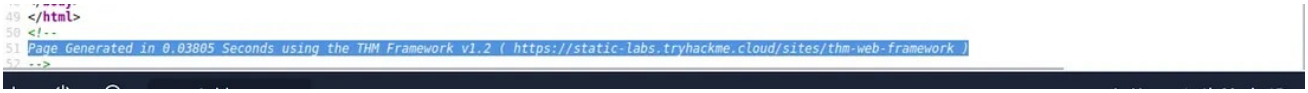


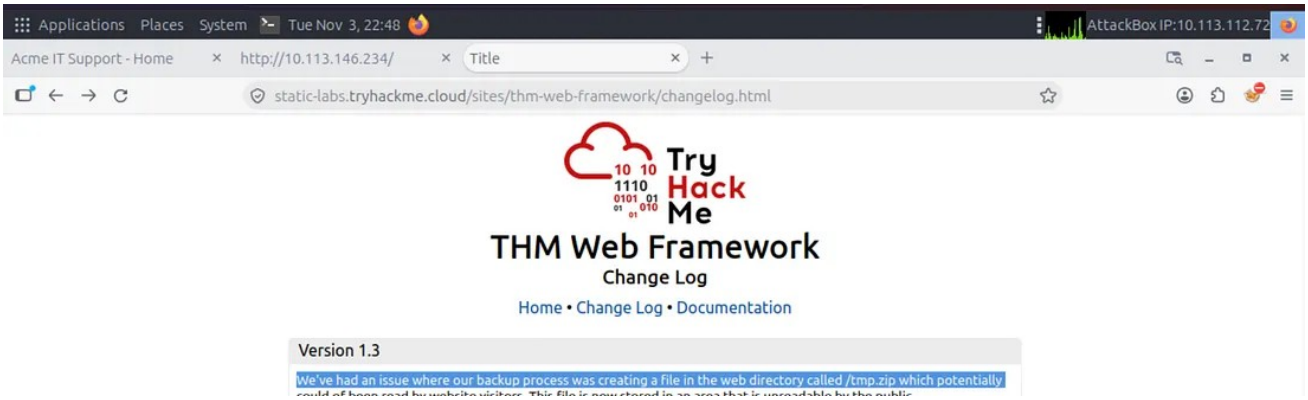


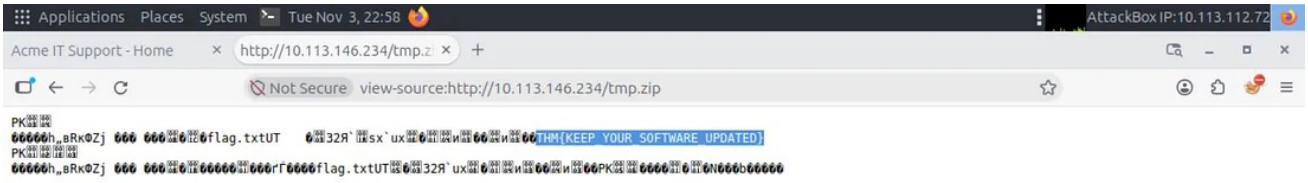
Q 5- What is the framework flag?

Answer : THM{KEEP_YOUR_SOFTWARE_UPDATED}

When we check the source code of the home page, we see a reference at the very bottom — “https://static-labs.tryhackme.cloud/sites/thm-web-framework” — indicating the presence of a `/tmp.zip` file, which contains backups from older versions. We navigate to `/tmp.zip` and extract the `flag.zip` file found inside.







Task 4-Developer Tools-Inspector

Developer Tools

Every modern browser includes developer tools; this is a toolkit used to aid web developers in debugging web applications and gives you a peek under the hood of a website to see what is going on. As pentesters, we can leverage these tools to gain a much better understanding of the web application. We're specifically focusing on three features of the developer toolkit: **Inspector, Debugger and Network.**

Opening Developer Tools

The way to access developer tools varies by browser. If you're not sure how to access it, click the `View Site` button on the top-right of this task to get instructions for accessing the tools for your browser.

Inspector

The page source doesn't always reflect what's shown on a webpage; CSS, JavaScript, and user interaction can change the page's content and style, so we need a way to view what's been displayed in the browser window at this exact time. The **Inspector** tab provides a live view of what is currently on the website. In addition to viewing this live view, we can also edit and interact with page elements, which is helpful for web developers to debug issues.



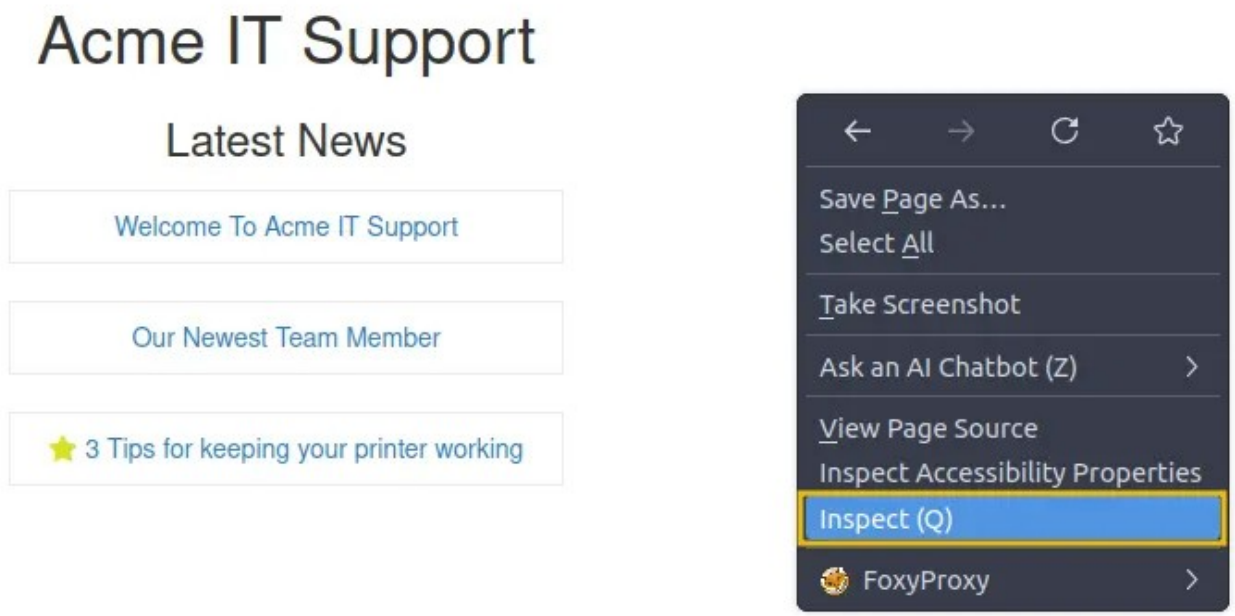
Subscribe to Pro Writer Tools to remove upgrade prompts from your unlocked stories.

On the Acme IT Support website, click into the **News** section, where you'll see three news articles. The first two articles are readable, but the third is blocked by a floating notice above the content stating that you need a premium subscription to view it. These floating boxes that block page content are often called **paywalls**, as they put up a metaphorical wall in front of the content you want to see until you pay.

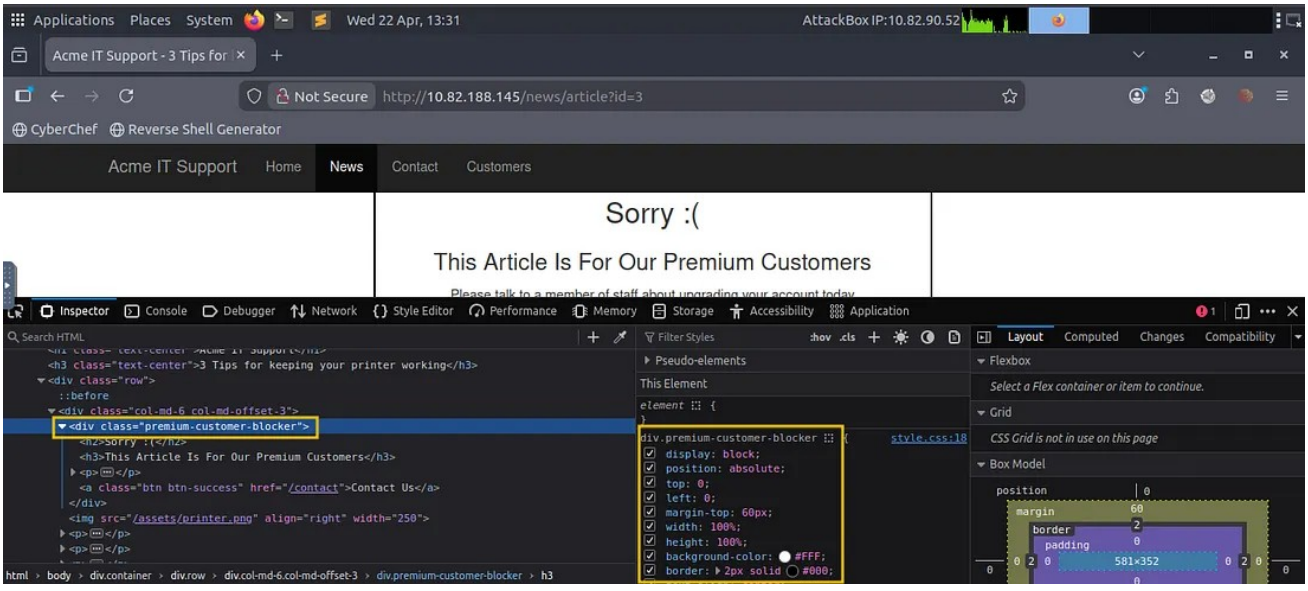


Right-click the premium notice (paywall), then select **Inspect** from the menu to

open the developer tools at the bottom or right-hand side, depending on your browser or preferences.



In the Inspector tab, you'll now see the elements/HTML that make up the website.



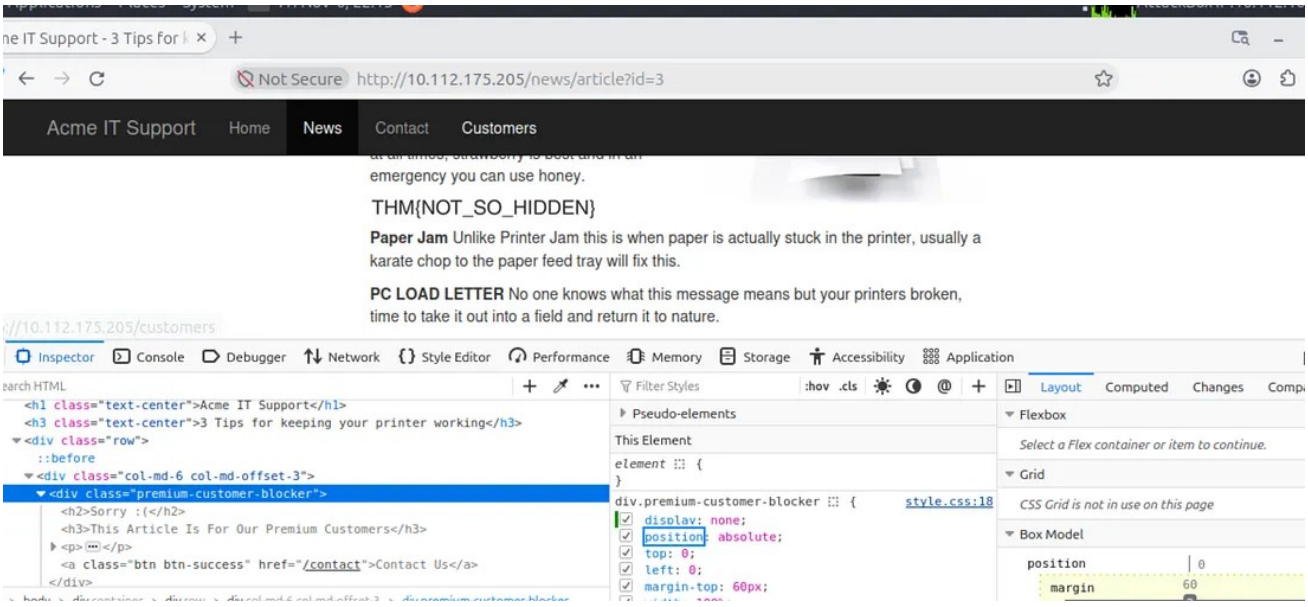
Locate the `DIV` element with the class `premium-customer-blocker`, and then click on it. You'll see all the CSS styles in the styles box that apply to this element, such as `margin-top: 60px` and `text-align: center`. The style we're interested in is the `display: block`. If you click on the word `block`, you can type a value of your own choice. Try typing `none`, and this will make the box disappear, revealing the

content underneath it and a **flag.** If the element didn't have a display field, you could click the bottom of the last style and add your own.

Have a play with the element inspector, and you'll see you can change any of the information on the website, including the content. Remember, this is only edited in your browser window, and when you press refresh, everything will be back to normal.

Q6-What is the flag behind the paywall?

Answer : THM{NOT_SO_HIDDEN}



Task 5- Developer Tools — Debugger

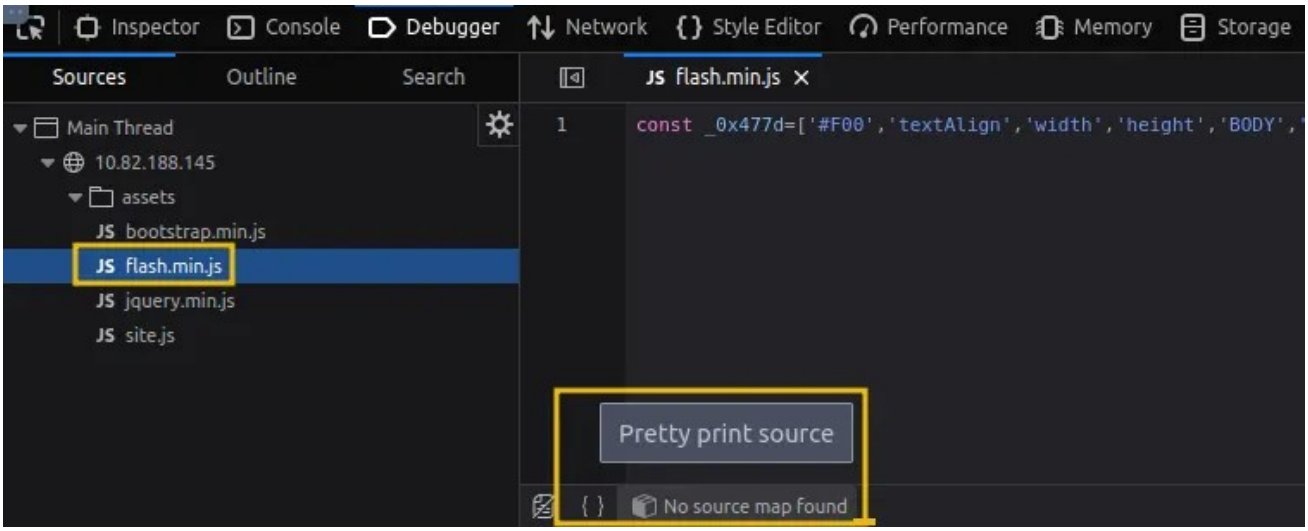
Developer Tools — Debugger

This panel in the developer tools is intended for debugging JavaScript, and again is an excellent feature for web developers wanting to work out why something might not be working. But as penetration testers, we can dig deep into the JavaScript code. In Firefox and Safari, this feature is called **Debugger**, but in Google Chrome, it's called **Sources**.

On the Acme IT Support website, click on the **Contact** page. Each time the page loads, you might notice a rapid flash of red on the screen. We're going to use the

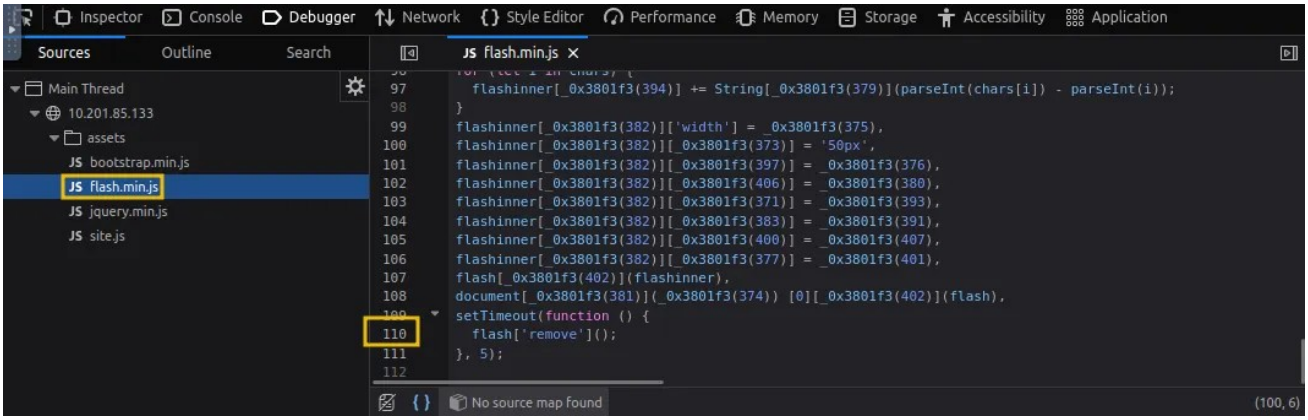
Debugger tab to work out what this red flash is and if it contains anything interesting. Debugging a red dot wouldn't be something you'd do in the real world as a penetration tester, but it does let us use this feature and get familiar with the Debugger.

In the Debugger tab, on the left-hand side, you see a list of all the resources the current webpage is using. If you click into the `assets` folder, you'll see a file named `flash.min.js`. Clicking this file displays its contents.



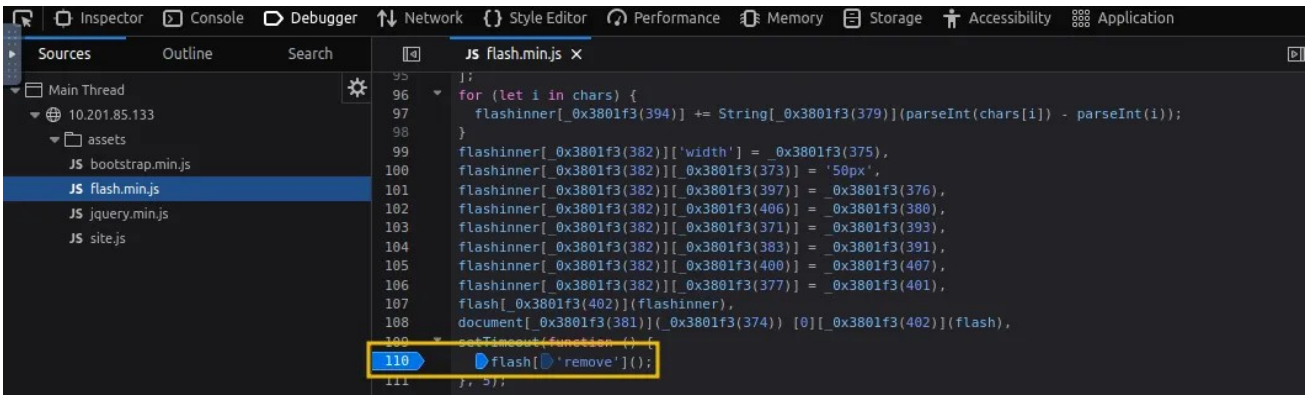
Often when viewing JavaScript files, you'll notice that everything is on one line because they've been minimised, meaning all formatting (tabs, spacing, and newlines) has been removed to make the file smaller. This file is no exception; it has also been obfuscated, making it purposely difficult to read and harder for other developers to copy.

We can return some of the formatting by using the **Pretty Print** option, which looks like two braces `{ }` to make it a little more readable, though due to the obfuscation, it's still difficult to comprehend what is going on in the file. If you scroll to the bottom of the `flash.min.js` file, you'll see the line: `flash['remove']();`.



This little bit of JavaScript is what is removing the red pop-up from the page. We can utilise another feature of the Debugger called **breakpoints**. These are code points we can use to force the browser to stop processing JavaScript and pause the current execution.

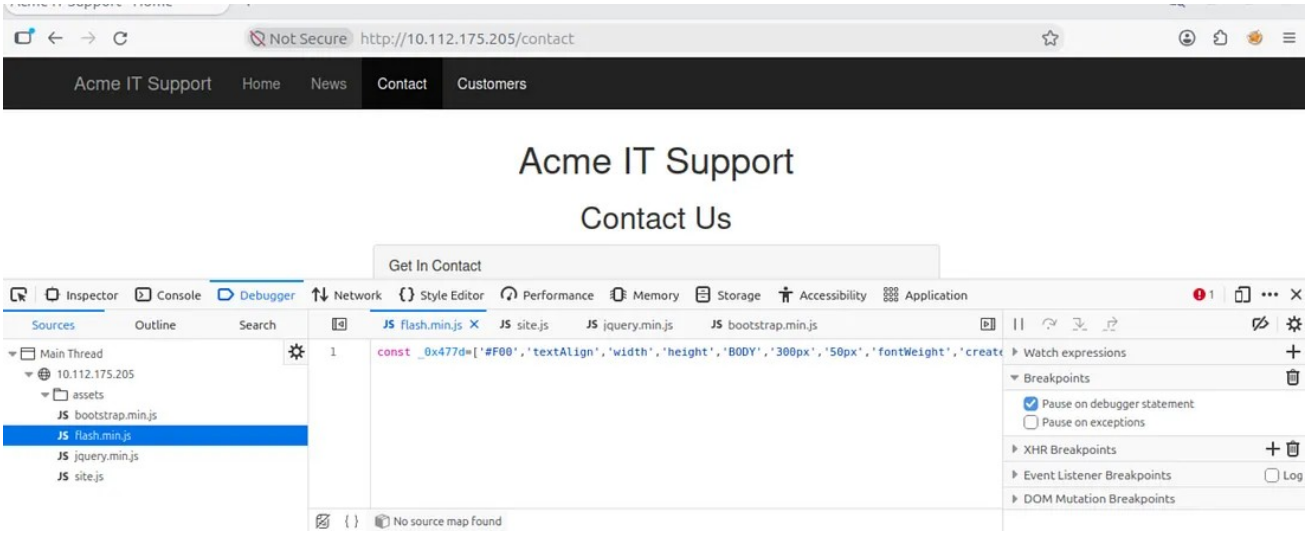
If you click line `110` that contains the code above, you'll notice it turns blue.

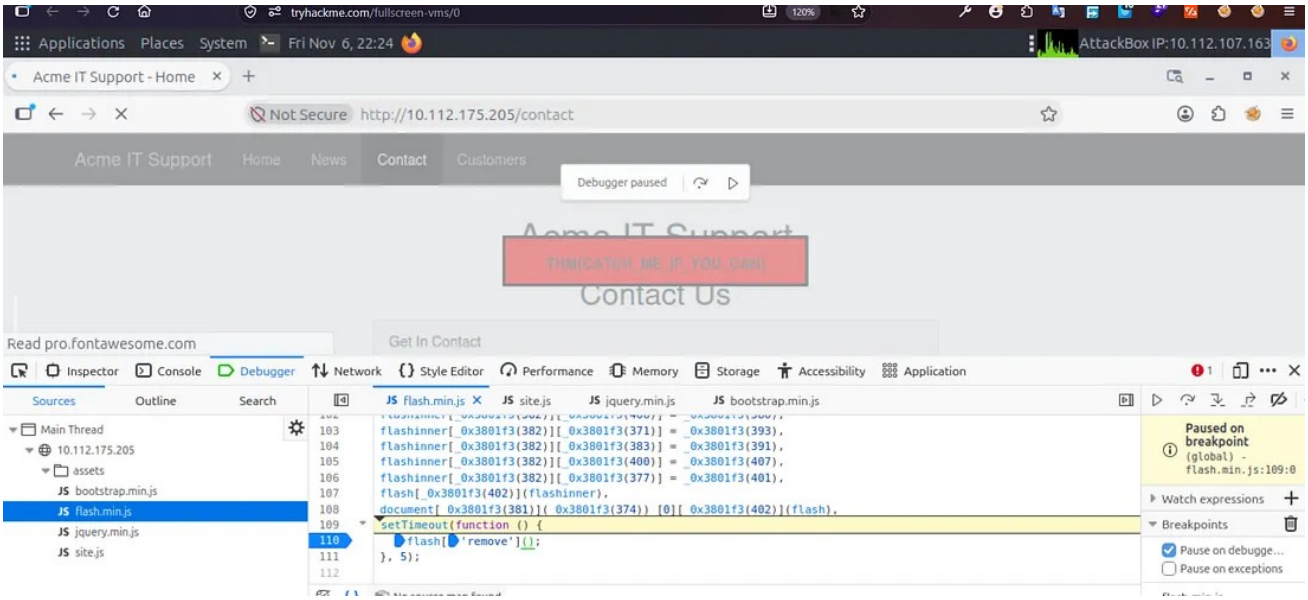


You've now inserted a breakpoint on this line. Now try refreshing the page, and you'll notice the red box stays on the page instead of disappearing, and it **contains a flag.**

Q 7-What is the flag in the red box?

Answer : THM{CATCH_ME_IF_YOU_CAN}



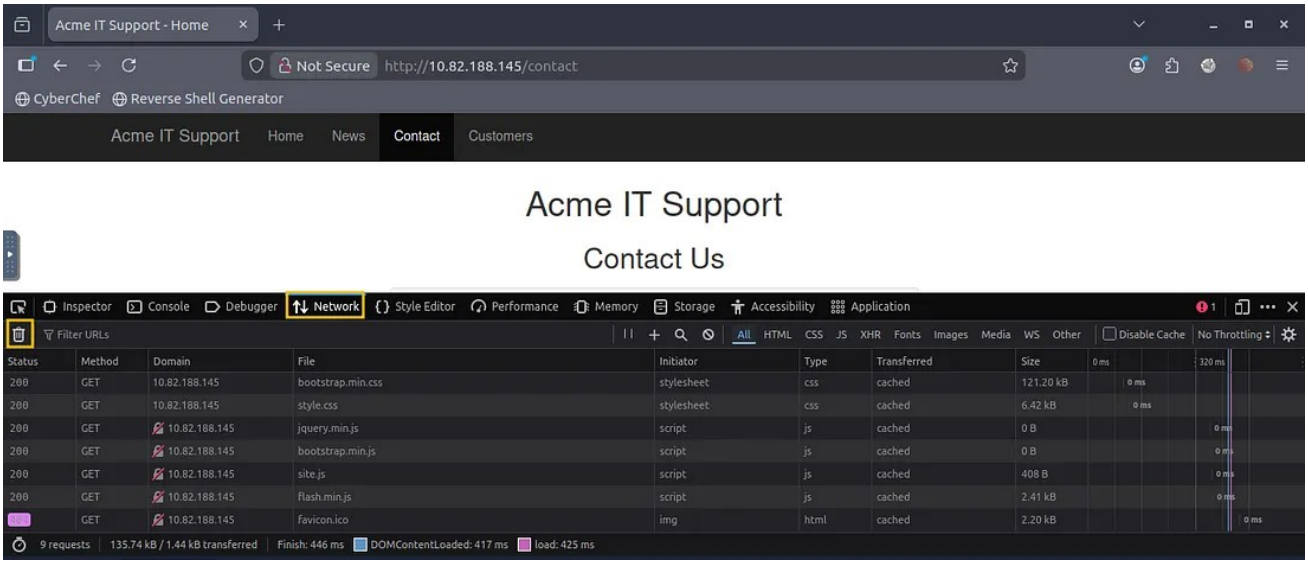


Task 6- Developer Tools — Network

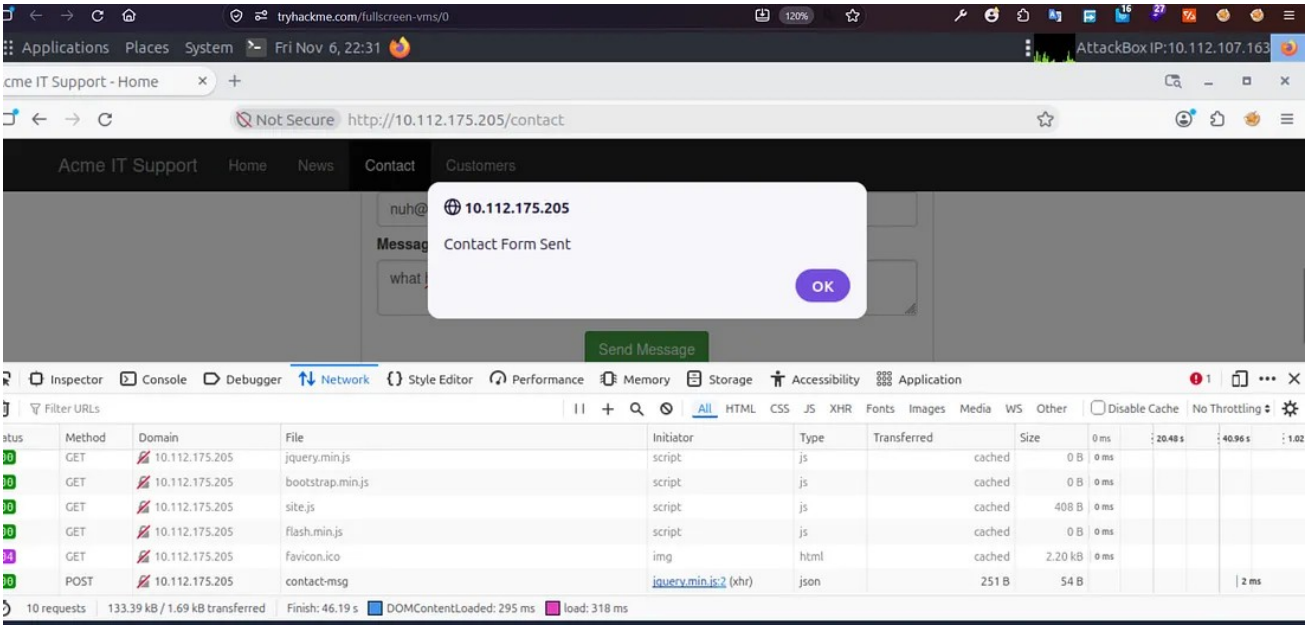
Developer Tools — Network

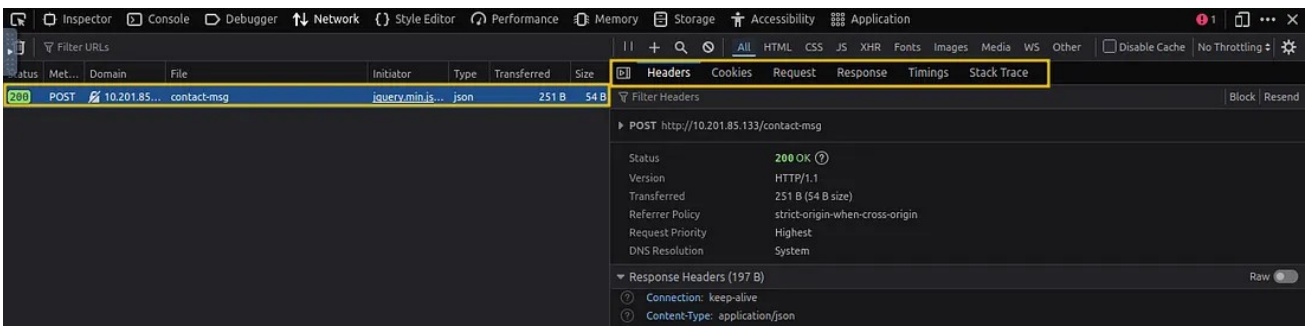
The **Network** tab in Developer Tools can be used to track every external request a webpage makes. If you click on the Network tab and refresh the page, you'll see all the files the page requests.

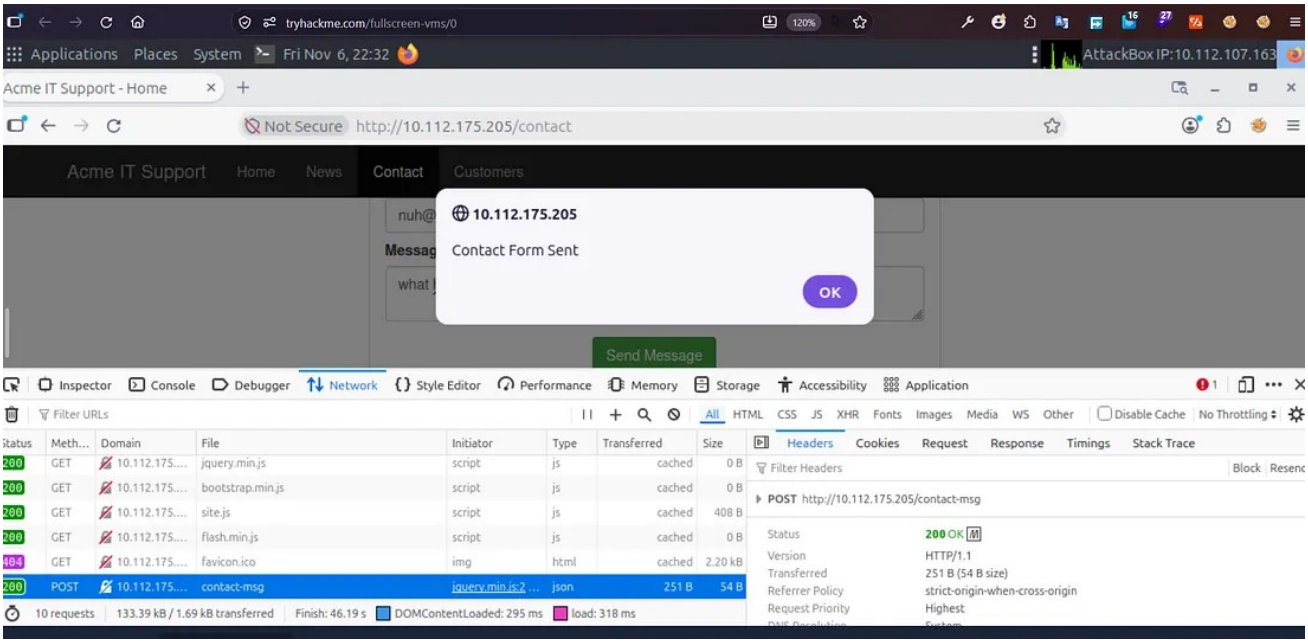
Try doing this on the **Contact** page; you can press the **trash can** icon to delete the list if it gets a bit overpopulated.



With the Network tab open, try filling in the contact form and pressing the **Send Message** button. You'll notice an event in the Network tab; this is the form being submitted in the background via AJAX. AJAX is a method for sending and receiving network data in a web application in the background without interfering with the current web page.



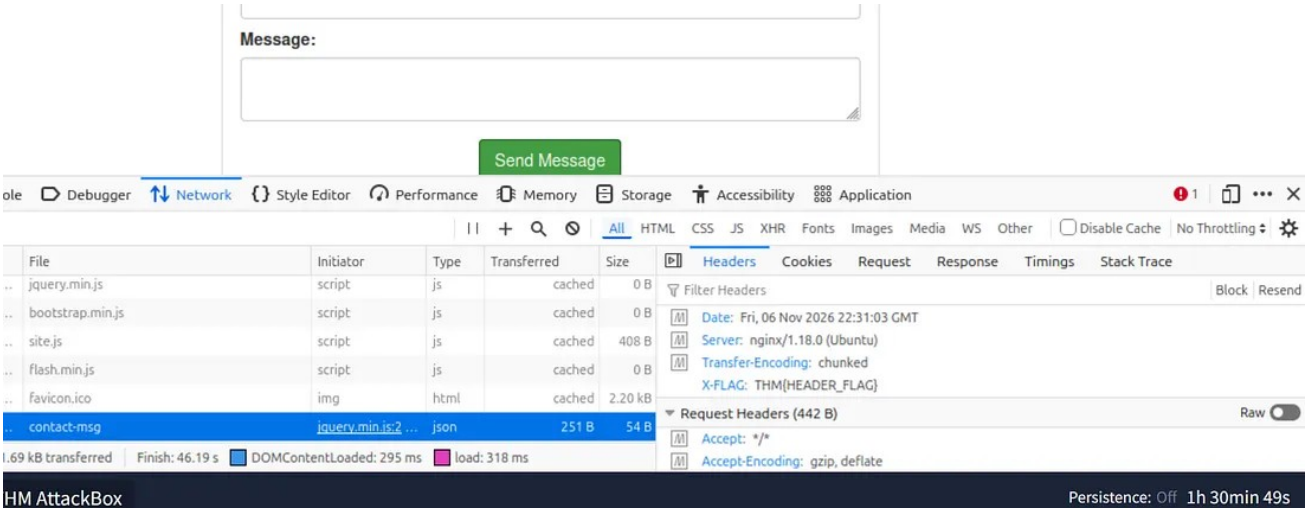


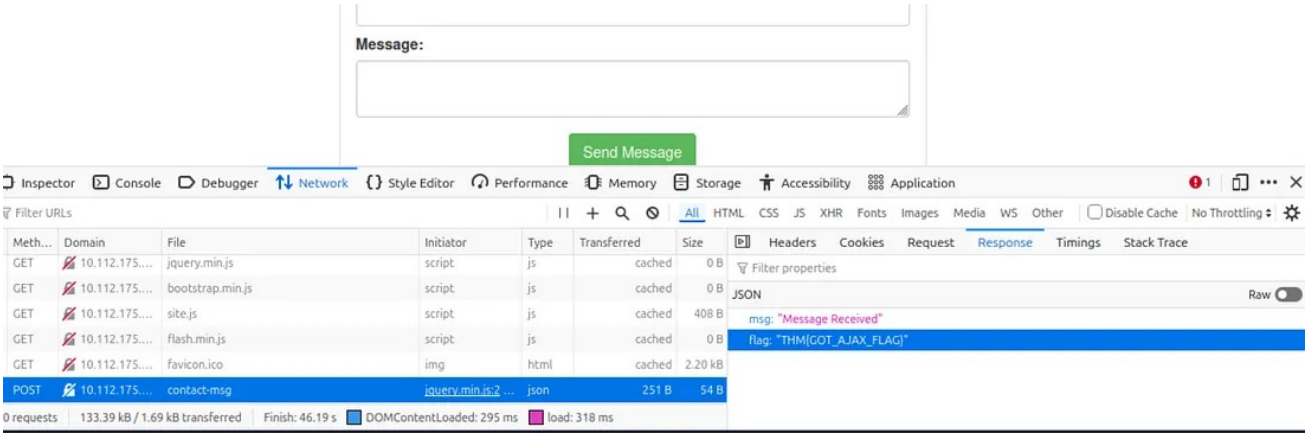


Once you examine the request, you will be able to see the request headers, cookie details and HTML response that would help you further help in enumeration and exploitation. Examine the new entry on the Network tab created by the contact form, and view the page the data was sent to **reveal a flag.**

Q 8-What is the flag under the Response tab on the contact-msg network request?

Answer : THM{GOT_AJAX_FLAG}





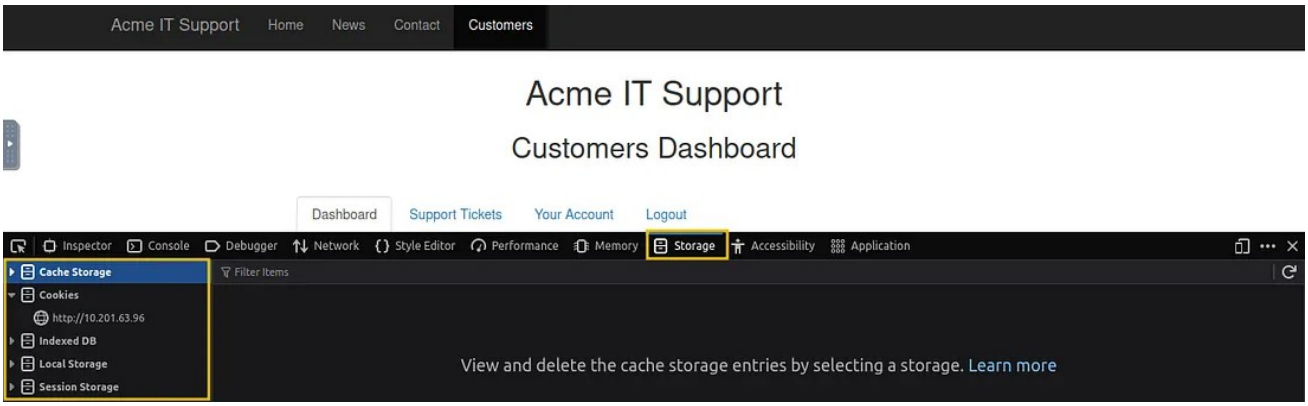
THM{GOT_AJAX_FLAG}

Task 7- Developer Tools-Storage

Storage Tab

The **Storage** tab in developer tools lets us view and manage data that a website stores in our browser. This data is stored on the client side and may contain sensitive or interesting information useful during a manual pentest. As pentesters, checking browser storage helps us understand how the application handles authentication, session data, user preferences, and other stored values.

On the webpage, create a new account by visiting `http://10.112.175.205/customers/signup`. After signing up, navigate to the Storage tab in developer tools, where you will see the following screen:

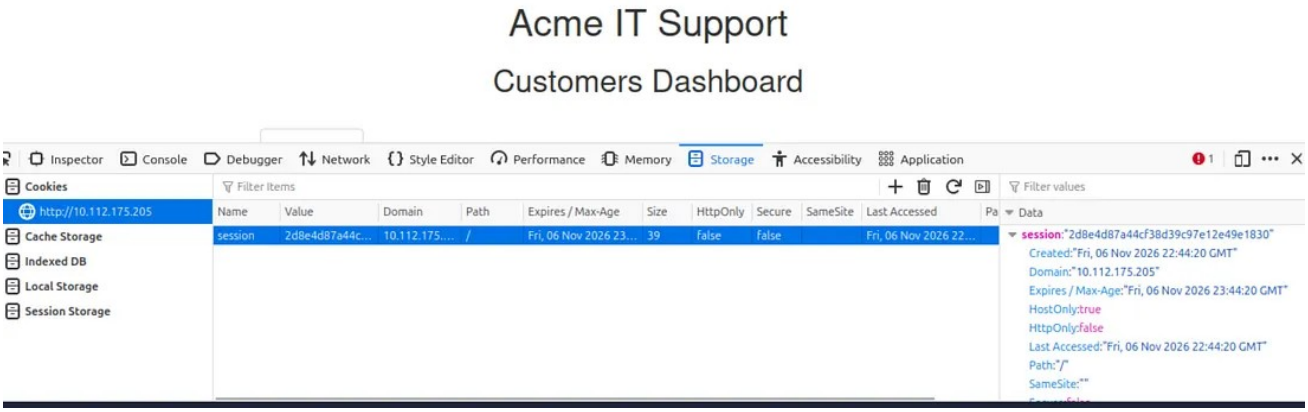


The Storage tab has the following important options:

- **Local Storage**: Stores data persistently in the browser, even after the browser is closed.

- **Session Storage**: Stores data temporarily for a single browser tab/session.
- **Cookies**: Small pieces of data sent by the server and stored in the browser, often used for sessions and authentication.
- **Cache Storage**: Stores cached resources like images, scripts, and API responses for faster loading

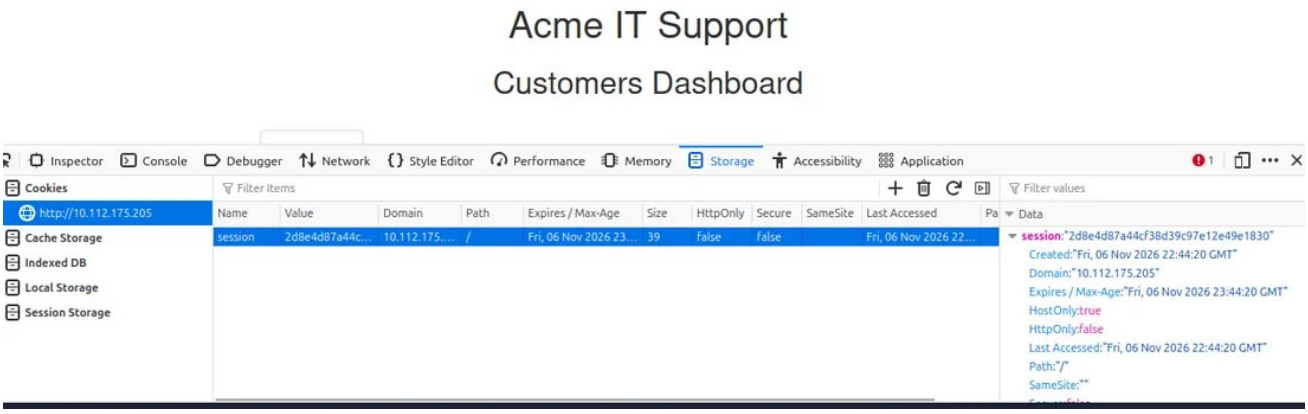
Among storage options, **cookies** are among the most important for a pentester. If you navigate to the **Cookies** section, you'll see the data stored on the client side by the website.



This often includes session identifiers, user preferences, and sometimes authentication-related tokens. Cookies also have important security flags. The `HttpOnly` flag prevents JavaScript from accessing the cookie, helping protect against XSS attacks. The `Secure` flag ensures the cookie is only sent over HTTPS, and the `SameSite` attribute helps mitigate CSRF attacks. Carefully reviewing cookies can reveal how the application manages sessions and whether any security best practices are missing.

Q 9-What is the value of the HttpOnly flag after logging in?

Answer : false



Task 8- Conclusion

In this room, we focused on manually assessing a web application using only the browser, no automated tools, no scanners, just observation and logical thinking. From **walking through an application** and **exploring the website** to analysing the **Page Source**, we learned how much information is openly exposed to anyone willing to look closely. Small details such as comments, hidden links, and misconfigurations can often reveal valuable insights.

By diving into the **developer tools**, including the **Inspector**, **Debugger**, **Network**, and **Storage** tabs, we explored how modern web applications function behind the scenes. We saw how content can be manipulated client-side, how network requests expose application behaviour, and how **cookies** and local storage handle session data. Understanding these components helps build a strong foundation in identifying client-side weaknesses.

Manual testing sharpens your pentesting mindset. Before relying on automated tools, learning to observe, question, and analyse what's happening in the browser will make you far more effective. Tools are powerful, but a trained eye and a curious mindset are even more powerful.

happy hacking

Nuh hozar - Medium

Read writing from Nuh hozar on Medium. Whatever is in the name of goodness and beauty, those who know us know, those...

nuhhozar.medium.com

nuh hozar block sayfası

https://nuhhozar.blogspot.com

nuhhozar.blogspot.com

NuhHozar - Overview

Only Curiosity for cyber world. NuhHozar has 36 repositories available. Follow their code on GitHub.

github.com

Walking An Application +



Follow

Written by Nuh hozar

8 followers · 12 following

Whatever is in the name of goodness and beauty, those who know us know, those who don't know know as well as themselves.!!